Lecture 10: Kernel II — Virtual Memory & Address Translation

Session 12 · Wed Oct 7, 2026

The Problem Virtual Memory Solves

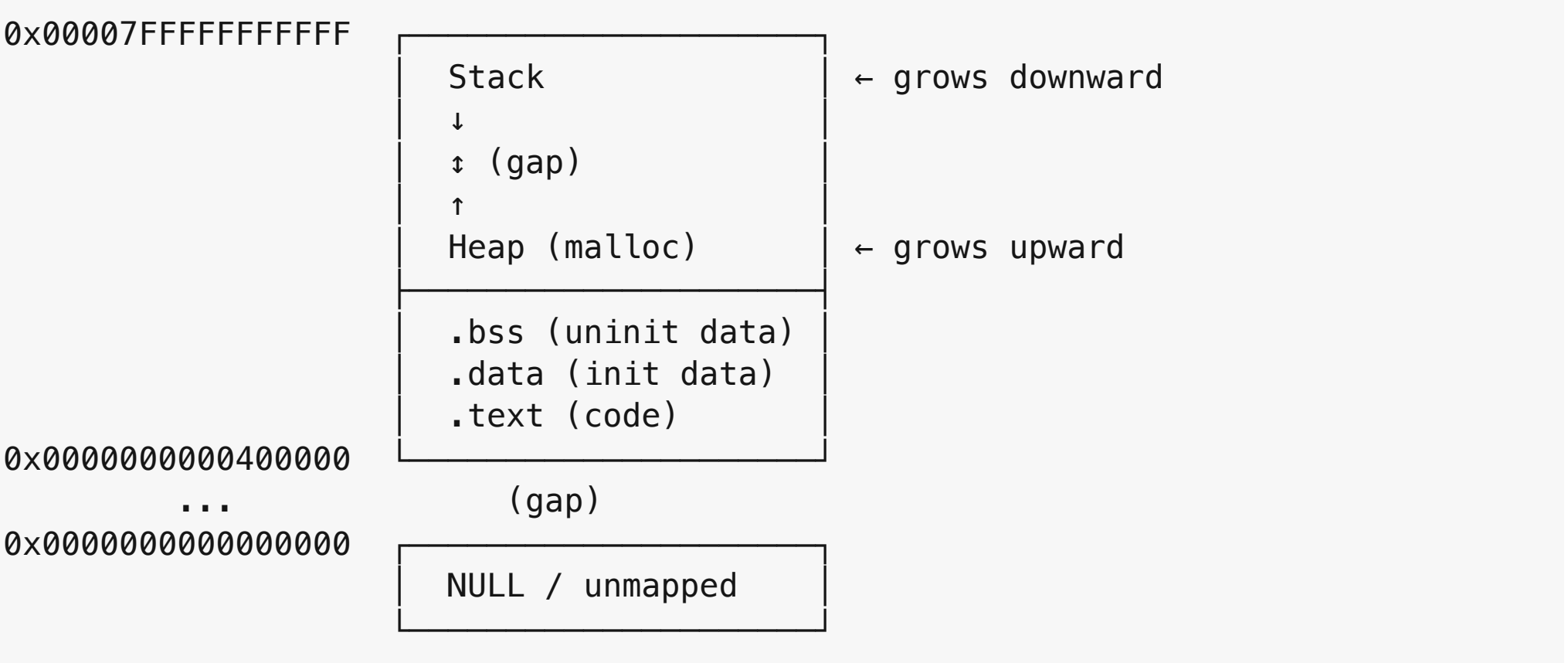
Lecture 9 said the kernel gives every process two illusions. The first is that the process has the whole CPU to itself, and Lecture 9 showed how the kernel does that. This lecture covers the second: each process believes it has all of memory to itself, so its addresses never collide with another program's. In reality all processes share one physical memory.

Without virtual memory, two programs that both use address **`0x1000`** collide in physical RAM. With virtual memory, each process has its own **virtual address space**, and the **Memory Management Unit** (MMU), a piece of hardware, translates every address the program uses into a physical one:

```
Without virtual memory:
  Process A:  address 0x1000 → physical RAM 0x1000
  Process B:  address 0x1000 → physical RAM 0x1000   ← collision!
With virtual memory:
  Process A:  address 0x1000 → [MMU] → physical 0xA000
  Process B:  address 0x1000 → [MMU] → physical 0xB000
```

Virtual Address Space Layout

Every process on Linux x86-64 sees the same map: code low, stack high, and the heap growing up toward the stack.



The gaps are addresses with no memory behind them. Touching one is the segmentation fault from Lecture 9.

Pages

The MMU does not translate one byte at a time. Memory is divided into **pages**, 4096 bytes each on x86-64, and translation works on whole pages:

```
getconf PAGE_SIZE   # 4096
```

Every allocation the kernel makes, every **`mmap`**, and every growth of the stack is a whole number of pages. The page-fault demo in Lecture 9 touched 16384 pages and paid 16383 page faults: one per page, never one per byte.

Address Translation

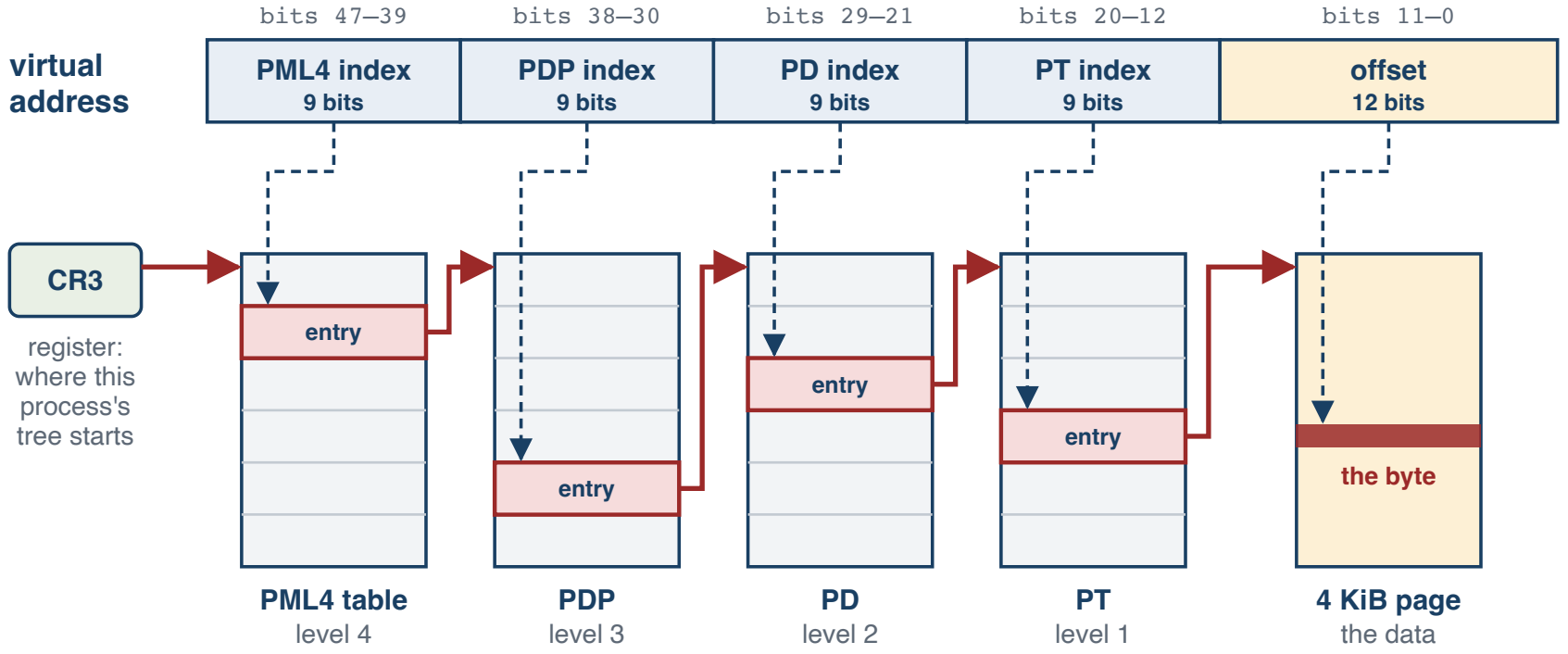
Every memory access the program makes, whether a load, a store, or an instruction fetch, goes through the MMU:



The MMU needs to know which physical page each virtual page maps to. That mapping is stored in the **page tables**, a tree of tables in memory that the kernel fills in and the MMU reads. Each process has its own tree, which is what makes its address space private.

Page Table Walk

x86-64 splits a 48-bit virtual address into four 9-bit indexes and a 12-bit offset. Each index selects an entry in one level of the tree:



Each table is one 4 KiB page: 512 entries of 8 bytes. On a process switch the kernel loads a new value into CR3.

The 12-bit offset is the position inside the 4096-byte page, and it is copied through unchanged.

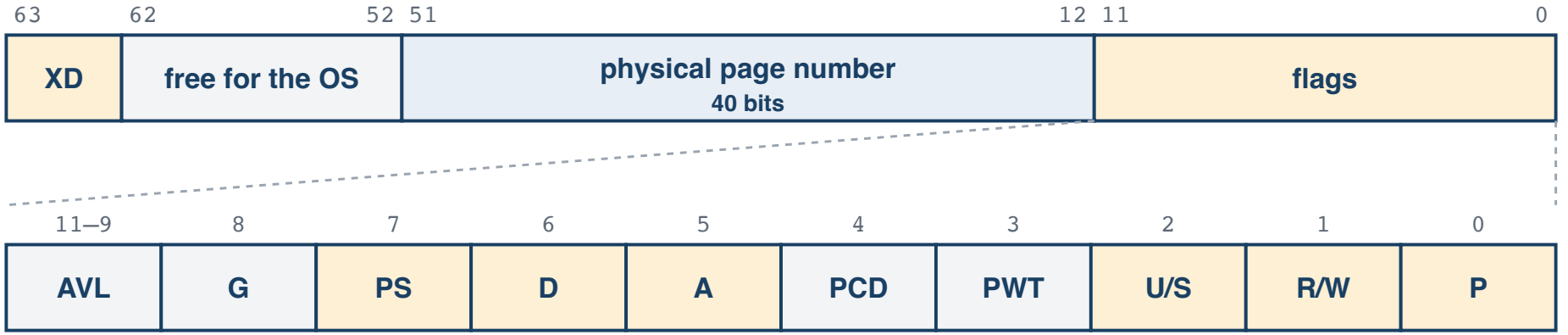
The Four Tables and Their Names

The names come from Intel's manuals. Every table is one 4096-byte page holding 512 entries of 8 bytes each, which is why each index is 9 bits ($2^9 = 512$).

Level	Name	Stands for	Its entry points to
4 (top)	PML4	Page Map Level 4	a PDP table
3	PDP	Page Directory Pointer table	a page directory
2	PD	Page Directory	a page table
1 (bottom)	PT	Page Table	the physical page

The CPU's **CR3** register holds the physical address of the current process's PML4. When the kernel switches to another process, it loads that process's PML4 address into **CR3**, and from then on every address is translated through the other process's tables.

Inside One Page Table Entry



P	Present: 0 means any use of the page faults	**XD**	Execute Disable: 1 means no instructions run from here
R/W	1 means the page may be written	**PS**	Page Size: in a PD entry, 1 means a 2 MB huge page
U/S	1 means user programs may use the page	**G**	Global: keep this entry in the TLB across CR3 changes
A	Accessed: the CPU sets it when the page is used	**PCD, PWT**	cache settings for the page
D	Dirty: the CPU sets it when the page is written	**AVL**	free for the operating system to use

bits this lecture uses

The physical page number says where the next table, or the page itself, starts in RAM. The kernel writes P, R/W, U/S and XD. The CPU sets A and D by itself.

The Latency Problem

Walking the tree costs memory accesses. To read one byte, the MMU must:

1. Look up the PML4 entry
2. Look up the PDP entry
3. Look up the PD entry
4. Look up the PT entry
5. Access the data

That is four extra memory accesses for every real one. If nothing were cached, every program would run about five times slower than it does.

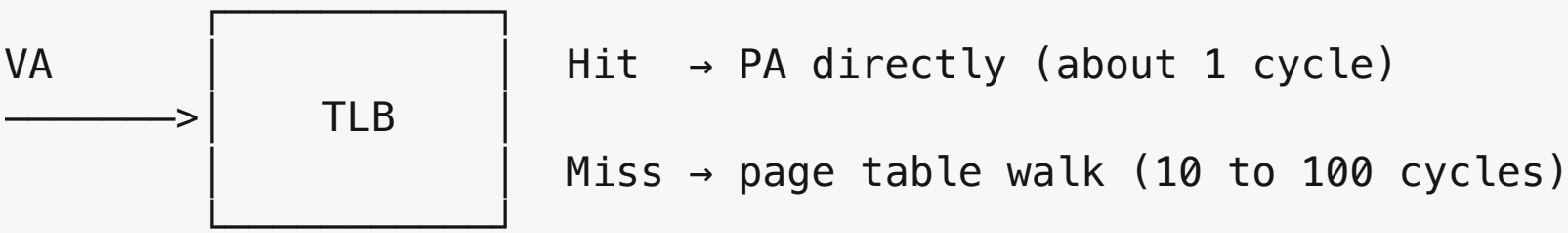
Fun Fact: Why 48 Bits and Not 64?

Pointers on x86-64 are 64 bits wide, but the MMU uses only the low 48 bits of a virtual address. 2^{48} bytes is 256 TiB, 4096 times the RAM in a 64 GB laptop, so no program needs more today. Using more bits has a cost: every 9 more bits needs one more level in the page-table tree, and so one more memory access each time the walk runs.

The top 16 bits must be copies of bit 47. If they are not, the CPU refuses the address and the kernel ends the program with a segmentation fault. Linux gives user programs the half where bit 47 is 0, which is 128 TiB. That is why the stack on the earlier slide starts at **`0x00007FFFFFFFFFFF`**.

Translation Lookaside Buffer (TLB)

The **TLB** is a small hardware cache inside the CPU that remembers recent translations, so most accesses skip the walk entirely:



A modern CPU's TLB holds somewhere between 64 and about 1500 entries, is 4-way or 8-way set-associative, and answers a hit in about one cycle. A miss costs a walk, which may itself miss in the data caches.

Huge Pages

A regular page is 4 KB, so a 1 GB mapping needs 262,144 page table entries, far more than any TLB holds. **Huge pages** of 2 MB or 1 GB cover the same memory with far fewer entries, and they shorten the walk by one or two levels:

```
2 MB page: 21 offset bits, 27 index bits → 3 levels (PML4→PDP→PD→page)
1 GB page: 30 offset bits, 18 index bits → 2 levels (PML4→PDP→page)
```

The page ends at whichever level stops pointing at another table. With 2 MB pages the page-directory entry maps the page itself, so the fourth table disappears.

```
grep Huge /proc/meminfo                          # is the machine using them?
mmap(NULL, 2 << 20, PROT_READ | PROT_WRITE,
     MAP_PRIVATE | MAP_ANONYMOUS | MAP_HUGETLB, -1, 0);   # ask for one
```

Page Faults, Seen From the Page Table

Now the page fault from Lecture 9 has a mechanism. When the MMU finds an entry whose Present bit is clear, or whose permission bits forbid the access, it raises exception 14 (a page fault: the CPU gives every kind of exception a number, and 14 means the MMU could not translate this address for this access), and the CPU jumps to the kernel's page-fault handler.

The handler looks at the address. If the process is allowed to use it, the kernel finds a free physical page, writes it into the page table entry, sets Present, and returns to the same instruction, which now succeeds. That is how **`malloc`** memory appears only when it is first touched. If the process is not allowed to use the address, the kernel sends **`SIGSEGV`**.

`SIGSEGV` is a **signal**, a message the kernel delivers to a process to tell it that something happened. SEGV stands for "segmentation violation", the old name for using an address the process is not allowed to use. A program can choose to handle a signal, but almost none do, so the kernel ends the program and the shell prints **`Segmentation fault (core dumped)`**. This is the same pattern as dividing by zero in Lecture 9: the CPU raises an exception, and the kernel turns it into a signal for the process. Lecture 19 covers signals in full.

Run It Again: Lecture 9's Page Faults

Lecture 9's **`02-page-faults`** wrote one byte into each page of 64 MiB from **`malloc`**:

```
pages touched      16384
minor page faults  16383
```

Now each step has a name. **`malloc`** reserved 16384 pages of virtual addresses, but their page table entries have Present clear. On the first write to each page:

1. The MMU walks the tree, finds Present clear, and raises exception 14.
2. The kernel takes a free physical page, fills it with zeros, writes its number into the entry, and sets Present.
3. The CPU runs the same write again, and this time the walk succeeds.

One page did not fault because **`malloc`** had already written its own bookkeeping into it. If the loop runs a second time over the same pages, every entry already has Present set, so nothing faults:

```
second pass faults 0                  # same program, loop run twice, Linux
```

mmap: Memory Mapping

mmap is the system call that creates a mapping: it asks the kernel to attach a range of virtual addresses to either a file or to fresh zeroed memory.

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

An **anonymous** mapping has no file behind it. This is how **malloc** gets large blocks from the kernel (Lecture 11):

```
void *mem = mmap(NULL, 1 << 20,                    // 1 MB of zeroed memory
                 PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
if (mem == MAP_FAILED) { perror("mmap"); exit(1); }
// ... use mem ...
munmap(mem, 1 << 20);
```

mmap: Mapping a File

A **file** mapping makes the bytes of a file appear in memory. The program reads them with ordinary pointer operations, and the kernel reads the file into pages on first touch, one page fault at a time:

```
int fd = open("data.bin", O_RDONLY);
struct stat st;
fstat(fd, &st);

char *data = mmap(NULL, st.st_size, PROT_READ,     // map the whole file
                  MAP_PRIVATE, fd, 0);
// data[0], data[1], ... are the file's bytes; no read() call needed
munmap(data, st.st_size);
close(fd);
```

> **Tip**
>
> This is how the kernel loads executables and shared libraries: it maps the file and lets page faults pull in only the parts that run.

mmap: Shared Memory

With **MAP_SHARED**, two processes that map the same memory see each other's writes. After **fork**, the child inherits the parent's mappings, so a shared anonymous mapping made before the fork is a channel between them:

```
void *shm = mmap(NULL, 4096, PROT_READ | PROT_WRITE,
                 MAP_SHARED | MAP_ANONYMOUS, -1, 0);
pid_t pid = fork();
if (pid == 0) {
    strcpy(shm, "hello from child");       // child writes
} else {
    wait(NULL);
    printf("Got: %s\n", (char *)shm);      // parent reads the same page
}
```

Lecture 17 covers **fork** and **wait** themselves.

Copy-on-Write

`fork` copies a whole process, which sounds expensive. The kernel avoids the copy with **copy-on-write**:

1. Parent and child share the **same physical pages**.
2. The kernel marks every shared page read-only in both page tables.
3. When either process writes to a page, the MMU raises a page fault. The kernel copies that one page, points the writer's entry at the copy, and marks both entries writable again.

Pages that are never written are never copied. That is why `fork` is fast even for a process using gigabytes.

Copy-on-Write in the Page Tables

```
Parent           Page tables           Physical memory
  VA 0x1000 ──> [PTE: PA 0xA000, r/o] ──> [page data]
                                              ▲
  VA 0x1000 ──> [PTE: PA 0xA000, r/o] ────────┘
Child

  // Child writes to VA 0x1000 → page fault

Parent           Page tables           Physical memory
  VA 0x1000 ──> [PTE: PA 0xA000, r/w] ──> [page data (original)]

  VA 0x1000 ──> [PTE: PA 0xB000, r/w] ──> [page data (copy)]
Child
```

Both processes still use virtual address **`0x1000`**. Only the physical page behind it differs after the write.

Memory Protection

The permission bits in each page table entry are set by the **`prot`** argument to **`mmap`**, and the MMU enforces them on every access:

```
// Read-only page: a write raises a page fault and the kernel sends SIGSEGV
mmap(NULL, 4096, PROT_READ, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

// Executable page, the kind a JIT compiler writes machine code into
mmap(NULL, 4096, PROT_READ | PROT_EXEC, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);

// No-access guard page: any touch is a fault, so an overflow is caught at once
mmap(NULL, 4096, PROT_NONE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
```

This is also what stops the buffer-overflow attack from Lecture 8 from running code on the stack: the stack's pages are mapped without **`PROT_EXEC`**.

Inspecting Process Memory

Linux lists every mapping of a process in **`/proc/<pid>/maps`**. The file **`/proc/self/maps`** is the map of whichever process reads it:

```
cat /proc/self/maps
```

```
00400000-00401000 r-xp ... /usr/bin/cat     # code
00600000-00601000 r--p ... /usr/bin/cat     # read-only data
00601000-00602000 rw-p ... /usr/bin/cat     # read-write data
7f1234000000-7f1234021000 rw-p ...          # heap
7fff12340000-7fff12360000 rw-p ... [stack]
```

Each line is one mapping: its address range, its permissions (**`r`**, **`w`**, **`x`**, and **`p`** for private or **`s`** for shared), and the file behind it, if any.

AI Systems Connection

Two `mmap` behaviours explain things people find mysterious about large models.

A 40 GB model "loads" in under a second. Tools like `llama.cpp` map the weight file with `mmap` instead of reading it. Nothing is copied. The kernel records that this range of addresses corresponds to that file, and pages arrive on demand as the model touches them. The load is instant and the first inference pass is slow, because that is when the reading really happens. It also means several processes running the same model share one copy of the weights in physical memory. Four workers using `read` need four times the RAM; four workers using `mmap` do not.

Pinned memory. Copying host memory to a GPU is done by a DMA engine, which needs the physical pages to stay put. So frameworks offer "pinned" buffers that the kernel will never move or swap out. They transfer faster, and they are scarce: pin too much and the rest of the system starts thrashing.

AI Systems Connection II

The TLB is small, on the order of a thousand entries, and a 4 KB page maps 4 KB. A model with 20 GB of weights needs about five million page table entries, and no TLB holds a meaningful fraction of that. Once the working set is that large, a noticeable share of memory accesses pay for a page table walk on top of the access itself.

This is what huge pages are for. One 2 MB page replaces 512 ordinary ones, so the same 20 GB needs roughly ten thousand entries instead of five million, and the walk itself is one level shorter. Linux offers transparent huge pages, and turning them on is routine when running large models.

The same idea reappears on the accelerator. A GPU has its own MMU and its own TLB, and unified-memory schemes that let the GPU fault on host pages are address translation across a bus, with everything in this lecture applying and the miss penalty far larger.

The Lecture in One Slide

- Each process has a private **virtual address space**. The **MMU** translates every address through the process's **page tables**, one **page** of 4096 bytes at a time.
- A translation walks four levels of tables. The **TLB** caches recent translations so most accesses skip the walk. **Huge pages** need fewer entries and a shorter walk.
- A **page fault** is the MMU finding no valid entry. The kernel either fills in the entry and retries the instruction, or ends the process.
- **mmap** attaches addresses to a file or to zeroed memory. **Copy-on-write** shares pages after `fork` and copies one only when it is written.

Practice

1. If a TLB has 64 entries and each covers a 4 KB page, how much memory can the program touch without a single TLB miss?
2. Why not use huge pages for everything?
3. After **`fork`**, parent and child see the same virtual addresses. Are they using the same physical memory? Does the answer change after one of them writes?
4. What happens if you **`mmap`** a page with **`PROT_NONE`** and then write to it? Name the exception number and the signal.
5. Write a program that maps a file with **`mmap`** and counts its newlines in one pass. Then print **`/proc/self/maps`** from inside the program and find your mapping.

Answers to Practice

1. 64 entries × 4 KB = 256 KB. A program whose working memory fits in 256 KB never misses in the TLB once the entries are loaded.
2. A huge page wastes memory when the program uses only a little of it: a 2 MB page holding 10 KB of data leaves almost all of it unused. The kernel also needs 2 MB of free physical memory in one contiguous block, which is hard to find on a busy machine. Each fault must zero 2 MB, and each copy-on-write must copy 2 MB, even for a one-byte write.
3. Right after **`fork`**, yes: both page tables point at the same physical pages, marked read-only. When one process writes, the kernel copies only that one page, so the two processes now have different physical pages behind that virtual address. Every other page stays shared.
4. The write raises exception 14, a page fault. The kernel sees that the process is not allowed to write there and sends **`SIGSEGV`**, which ends the program with a segmentation fault. (macOS sends **`SIGBUS`**, "Bus error", instead. **`06-protect.c`** shows both.)

Answers to Practice 5

```c
// Needs fcntl.h, stdio.h, string.h, sys/mman.h and sys/stat.h
int main(int argc, char **argv) {
    if (argc != 2) return 1;
    struct stat st;
    int fd = open(argv[1], O_RDONLY);
    fstat(fd, &st);                          // st.st_size: file length
    char *p = mmap(NULL, st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    long lines = 0;
    for (off_t i = 0; i < st.st_size; i++)  // the file is an array
        if (p[i] == '\n') lines++;
    printf("%ld newlines\n", lines);
    char buf[512];
    FILE *maps = fopen("/proc/self/maps", "r");
    while (fgets(buf, sizeof buf, maps))
        if (strstr(buf, argv[1])) fputs(buf, stdout);   // our file only
}
```

```
$ ./newlines /etc/services            # Linux
361 newlines
7b96b6b64000-7b96b6b68000 r--p 00000000 08:02 14418390   /etc/services
```

Reading

- CS:APP §9.1–9.8: Virtual Memory
- **`man mmap`**
- OSDev wiki on paging